

RESIDÊNCIA EM MICROELETRÔNICA — POLO UFCG



Design Verification

Verification Guide

Fase 2 – PARTE II

Projeto Integrador de Design Verification

guia de uso e debug do ambiente de verificação

Instrutor e Autor:
Gabriel Villanova

Aluno:
João Oliveira

Documento:
Verification Guide – Fase 2

CONFIDENCIAL
V1.0.0
Setembro/2026

Sumário

1 Introdução

2 Arquitetura do testbench

3 Estrutura do repositório

4 Dependências e setup

5 Compilação e execução

6 Testes, sequences e virtual sequences

7 Debug e waveforms

8 Regressão e coverage

9 Como estender o ambiente

10 Limitações e referências

“Hard work always pays off.”

1 Introdução

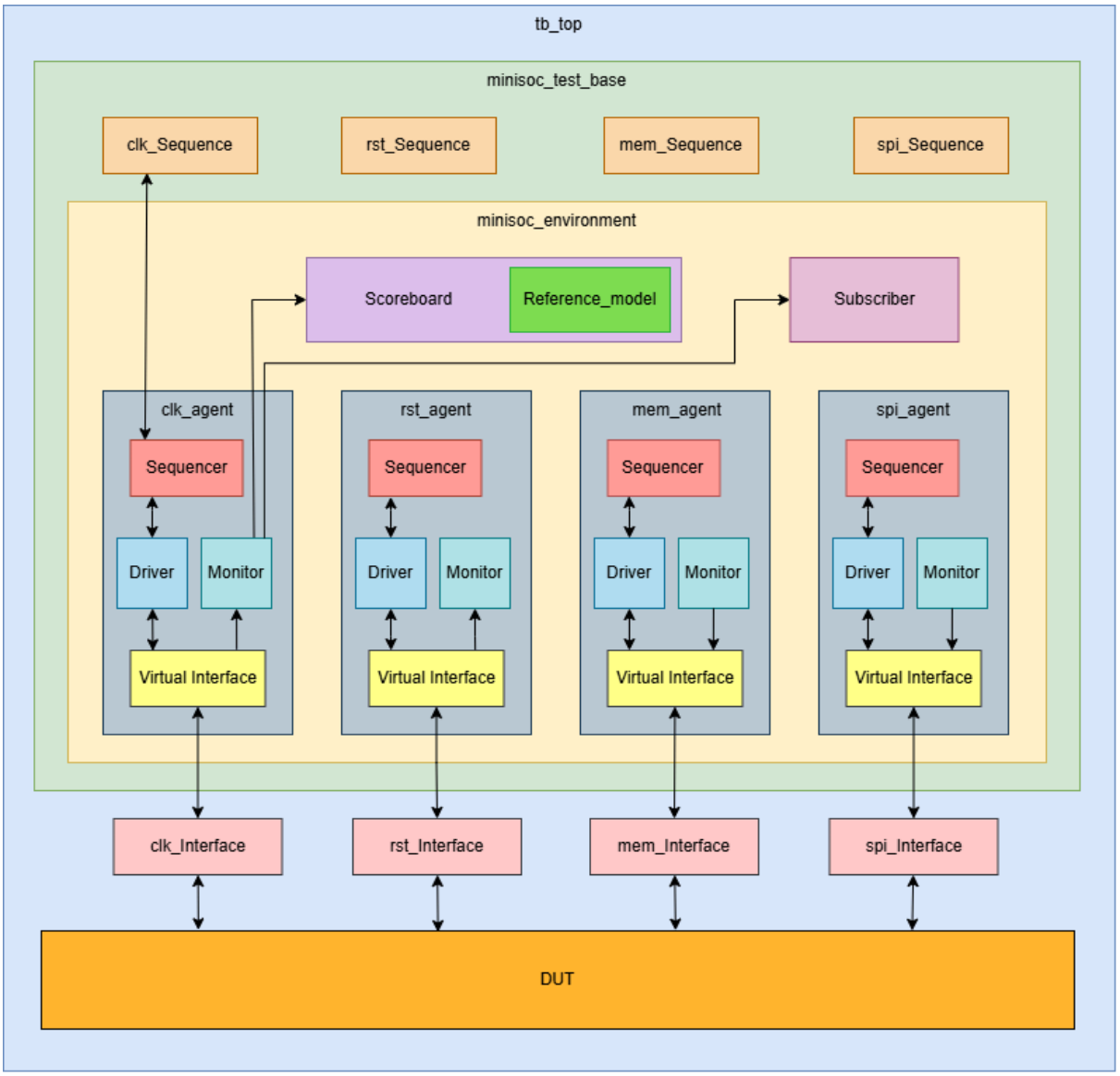
1.1 Propósito do documento

Este documento deve permitir que um novo usuário entenda a arquitetura do testbench, prepare o ambiente, compile o projeto, execute testes e regressões, colete coverage, abra waveforms, reproduza falhas por seed e localize os principais pontos de extensão.

1.2 Escopo e baseline documentado

Campo	Preencher com
DUT / projeto	[Nome do top-level / Fase 2 AES]
RTL tag/commit	[Tag ou hash exato]
TB tag/commit	[Tag ou hash exato]
Simulator	Synopsys VCS X-2025.06-SP2
UVM	UVM 1.1d
Sistema operacional	Rocky Linux 8.10 (Green Obsidian), kernel Linux 4.18.0-553.132.1.el8_10.x86_64

2 Arquitetura do testbench



"Hard work always pays off."

Componente	Responsabilidade	Interfaces / conexões
tb_top	Instancia o DUT, as interfaces e inicia a execução do ambiente UVM.	Conecta o DUT às interfaces clk_if, rst_if, mem_data_if e spi_if, além de iniciar o teste UVM.
minisoc_test_base	Define a estrutura base dos testes e instancia o ambiente de verificação.	Utiliza o minisoc_environment e organiza as sequências de clock, reset, memória e SPI.
minisoc_environment	Agrupa os agentes e os componentes responsáveis pela verificação funcional do MiniSoC AES.	Contém os agentes clk_agent, rst_agent, mem_data_agent e spi_agent, além do Scoreboard e do Subscriber.
clk_agent	Gerencia a geração e a observação dos sinais de clock.	Composto por clk_sqr, clk_drv, clk_mon e acesso à interface clk_if por meio da interface virtual.
rst_agent	Gerencia a aplicação e a observação dos sinais de reset.	Composto por rst_sqr, rst_drv, rst_mon e acesso à interface rst_if por meio da interface virtual.
mem_data_agent	Gerencia os estímulos e a observação das transações da interface de memória.	Composto por mem_data_sqr, mem_data_drv, mem_data_mon e acesso à interface mem_data_if por meio da interface virtual.
spi_agent	Gerencia os estímulos e a observação das transações SPI.	Composto por spi_sqr, spi_drv, spi_mon e acesso à interface spi_if por meio da interface virtual.
Sequencer	Gerencia as transações geradas pelas sequências e as disponibiliza ao driver correspondente.	Conecta-se ao driver do respectivo agente.
Driver	Recebe transações do sequencer e converte essas transações em estímulos aplicados ao DUT.	Acessa os sinais da interface correspondente por meio de uma interface virtual.
Monitor	Observa os sinais das interfaces e acompanha as transações realizadas durante a simulação.	Acessa a interface correspondente por meio de uma interface virtual e disponibiliza as informações observadas ao Scoreboard e Subscriber
Scoreboard	Verifica os resultados obtidos durante a simulação e identifica divergências em relação aos resultados esperados.	Integra-se ao fluxo de verificação e utiliza o modelo de referência AES por meio da interface DPI-C
Reference Model	Calcula os resultados esperados para as operações AES utilizadas na verificação.	Implementado em C/C++, acessível dentro do scoreboard do testbench por meio de DPI-C, incluindo a função c_aes_process().
Subscriber	Coleta informações para a análise da cobertura funcional da verificação.	Integrado ao ambiente UVM para receber as informações disponibilizadas pelo monitor
DUT	Representa o projeto RTL do MiniSoC AES submetido à verificação.	Conectado às interfaces de clock, reset, memória e SPI, por meio das interfaces

3 Estrutura do repositório

Path	Conteúdo	Quando o usuário precisa dele
verif/tb/	Top, interfaces e estrutura UVM.	Entender ou modificar o ambiente.
verif/tb/agents/	Agentes UVM de clock, reset, memória e SPI.	Modificar drivers e monitors.
verif/tb/env/	Ambiente, scoreboard e subscriber.	Integrar componentes e verificar resultados.
verif/tb/sequences/	Sequências de estímulo UVM.	Criar ou modificar estímulos.
verif/tb/tests/	Classes de teste UVM.	Criar ou selecionar cenários.
verif/tb/if/	Interfaces SystemVerilog.	Modificar conexões com o DUT.
verif/tb/refmod/	Modelo de referência AES em C/C++ e integração DPI.	Verificar resultados esperados.
verif/doc/	Documentação da verificação.	Consultar o plano de verificação.

4 Dependências e setup

CRITÉRIO DE QUALIDADE

Uma pessoa deve conseguir executar o primeiro smoke test seguindo esta seção. Registre versões reais; não escreva apenas “Python instalado” ou “simulador configurado”.

4.1 Dependências

Dependência	Versão / origem	Observação
Simulator	[versão]	[licença / módulo / container]
UVM	[versão]	[como é selecionada]
Python	[versão]	[se necessário]
Pacotes Python	[lista]	[requirements.txt se aplicável]
VIPs comuns	[nome + tag]	[dependências externas]

4.2 Environment setup

Forneça os comandos reais e em ordem. Exemplo de estrutura:

Passo	Comando	Resultado esperado
1	cd ~/<repo>/verif/tb	Entrar no diretório de execução do testbench.
2	source /Tools/synopsys-scripts/snps.sh	Carregar o ambiente e os caminhos das ferramentas Synopsys.
3	make	Compilar o projeto e executar o teste padrão
4	make vcs_comp	Compilar e elaborar o RTL, o testbench UVM e o modelo de referência C++, gerando o executável simv.
5	make vcs_run	Executar o teste padrão e coletar dados de cobertura.
6	make vcs_run UVM_TEST=<test>	Executar outro teste e coletar dados de cobertura.

5 Compilação e execução

O QUE DOCUMENTAR

Liste targets úteis, opções sobrescrevíveis, plusargs e exemplos completos. Um comando de exemplo deve ser copiável. Se houver caminhos absolutos, substitua por variáveis do projeto.

5.1 Make targets

Target	Descrição	Exemplo
all	Compila o projeto e executa o teste padrão.	make
vcs_comp	Compila e elabora o RTL, o testbench UVM e o modelo de referência C++, gerando o executável simv.	make vcs_comp
vcs_run	Executa o teste UVM selecionado e coleta dados de cobertura.	make vcs_run UVM_TEST=<test>
vcs_gui	Executa a simulação com a interface gráfica do VCS.	make vcs_gui
vcs_verdi	Abre o Verdi para visualizar as formas de onda do arquivo waves.fsdb.	make vcs_verdi
vcs_wave	Abre o visualizador nWave com as formas de onda do arquivo waves.fsdb.	make vcs_wave
clean	Remove executáveis, logs e arquivos temporários selecionados.	make clean

5.2 Opções, plusargs e variáveis

Nome	Função	Valores / exemplo
UVM_TESTNAME	Seleciona teste.	<class_name>
SEED	Reproduz randomização.	12345 / random
UVM_VERBOSITY	Controla log UVM.	UVM_LOW ... UVM_DEBUG
XRUN_ARGS	Argumentos adicionais do simulador.	[documentar]
WAVES	Habilita dump/probe se adotado.	0/1 ou target específico

5.3 Smoke test obrigatório

Comando recomendado como primeira validação do ambiente, com o tempo aproximado de execução e quais mensagens no log confirmam sucesso.

- *Comando:*

  make sim UVM_TESTNAME=<smoke_test> SEED=1

- *Tempo aproximado:*

  20 segundos

- *Mensagens no log de sucesso:*

UVM_INFO ./tests/minisoc_base_test.sv(35) @ 0: uvm_test_top [BASE_TEST] Starting execution of test: minisoc_rst_clk_test

UVM_INFO ./tests/minisoc_rst_clk_test.sv(52) @ 0: uvm_test_top [T1] T1 - Clock: verificando periodo do clock

UVM_INFO ./tests/minisoc_rst_clk_test.sv(67) @ 6000: uvm_test_top [T1] PASS: periodo medido = 0.500 ns

UVM_INFO ./tests/minisoc_rst_clk_test.sv(77) @ 6000: uvm_test_top [T2] T2 - Reset global: verificando assercao

UVM_INFO ./tests/minisoc_rst_clk_test.sv(90) @ 6000: uvm_test_top [T2] Reset ativo: rst_n = 0

UVM_INFO ./tests/minisoc_rst_clk_test.sv(94) @ 8000: uvm_test_top [T3] Reset liberado: rst_n = 1

UVM_INFO ./tests/minisoc_rst_clk_test.sv(100) @ 9000: uvm_test_top [T2] PASS: reset foi aplicado

UVM_INFO ./tests/minisoc_rst_clk_test.sv(106) @ 9000: uvm_test_top [T3] T3 - Reset: verificando liberacao

UVM_INFO ./tests/minisoc_rst_clk_test.sv(110) @ 9000: uvm_test_top [T3] PASS: reset foi liberado

UVM_INFO ./tests/minisoc_rst_clk_test.sv(116) @ 9000: uvm_test_top [TEST_SUMMARY] ========== RESUMO DOS TESTES ==========

UVM_INFO ./tests/minisoc_rst_clk_test.sv(117) @ 9000: uvm_test_top [TEST_SUMMARY] T1 - Clock: PASS

UVM_INFO ./tests/minisoc_rst_clk_test.sv(119) @ 9000: uvm_test_top [TEST_SUMMARY] T2 - Reset global: PASS

UVM_INFO ./tests/minisoc_rst_clk_test.sv(121) @ 9000: uvm_test_top [TEST_SUMMARY] T3 - Liberacao do reset: PASS

UVM_INFO ./tests/minisoc_rst_clk_test.sv(123) @ 9000: uvm_test_top [TEST_SUMMARY] ==

--- UVM Report Summary ---

*** Report counts by severity*

UVM_INFO : 19

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

VCS Coverage Metrics: during simulation line, cond, FSM, branch, tgl was monitored

---

Coverage status: End of All Coverages ...

V C S S i m u l a t i o n R e p o r t

Time: 8500 ps

6 Testes, sequences e virtual sequences

NÍVEL DE DETALHE

Descreva intenção e knobs importantes. Não copie o código-fonte das sequences. O leitor deve saber quando usar cada fluxo e quais parâmetros alteram seu comportamento.

6.1 Catálogo de testes

Teste	Objetivo	Fluxo principal	Quando usar
<smoke_test>	Sanidade do ambiente.	reset → config mínima → operação curta	Bring-up / CI rápido.
<aes_e2e_test>	Checagem AES fim a fim.	config → memory data → AES → compare	Funcional principal.
<reset_runtime_test>	Reset durante atividade.	tráfego → reset → recuperação	Robustez.
<random_test>	Explorar espaço legal.	variações constrained-random	Regressão/closure.

6.2 Sequences e knobs

Sequence	Responsabilidade	Knobs relevantes
<spi_config_seq>	Programa configuração via SPI.	[modo, endereço, valor...]
<memory_data_seq>	Gera operações read/write.	[tipo, endereço, payload, gaps...]
<reset_seq>	Aplica reset inicial/runtime.	[duração, momento...]
<virtual_seq>	Coordena múltiplas interfaces.	[cenário, repetições, ordem...]

7 Debug e waveforms

DEBUG REPRODUZÍVEL

Todo bug aleatório precisa de seed. Registre comando, seed, tag/commit e primeiro erro relevante. Não depure a partir do último erro em cascata se o log mostra um mismatch anterior.

7.1 Fluxo recomendado de debug

- Reproduzir exatamente com o mesmo test, seed e versão do código.
- Localizar o primeiro UVM_ERROR/UVM_FATAL/assertion/mismatch temporalmente relevante.
- Classificar: estímulo inválido, bug do TB, bug do RTL, limitação do modelo ou especificação ambígua.
- Abrir waveform apenas com sinais do caminho associado ao erro.
- Registrar causa-raiz e correção no histórico/issue do projeto.

7.2 Logs, probes e waveform setup

Artefato	Path / comando	Uso
Simulation log	[path]	Primeira fonte para failures.
Wave database	[path]	Visualização temporal.
Probe TCL	[path]	Seleciona sinais a gravar.
SimVision config/SVCF	[path]	Restaura grupos/janelas úteis.
Coverage DB	[path]	Análise de holes.

7.3 Falhas comuns

Sintoma	Causa provável	Primeira ação
m_config == null	Config_db/path/configuração do agent incorreta.	Verificar set/get e hierarquia.
Teste não inicia sequence	Phase/objection/default_sequence/virtual sequence.	Inspecionar flow do test.
Mismatch AES	Config/chave/payload/alinhamento ou latência.	Comparar transações no scoreboard/refmodel.
Sem waves	Probe/dump não carregado.	Revisar target e arquivo TCL.

8 Regressão e coverage

REPRODUTIBILIDADE

Explique onde a lista de testes/seeds vive, como lançar a regressão, onde os resultados são consolidados e como abrir o coverage. Não basta anexar uma captura de tela.

Ação	Comando / arquivo	Saída esperada
Rodar regressão	[make regression ...]	Resumo pass/fail por test/seed.
Reexecutar falha	[comando exato]	Mesmo erro reproduzido.
Merge coverage	[comando]	Coverage DB consolidado.
Abrir coverage	[comando GUI/HTML]	Relatório por covergroup/code/assertion.
Salvar evidência	[path]	Logs/summary/coverage versionados ou arquivados.

9 Como estender o ambiente

OBJETIVO

Ensinar o próximo engenheiro a adicionar uma feature sem quebrar a arquitetura. Use uma checklist curta e coerente com o VPLAN.

- Adicionar/atualizar feature e rastreabilidade no VPLAN.
- Implementar stimulus/sequence no agente apropriado.
- Adicionar observação/checker/refmodel necessários.
- Adicionar coverpoint/cross que meça a nova intenção funcional.
- Criar ou estender test/virtual sequence.
- Executar regressão de sanidade e depois closure da nova feature.
- Atualizar este Guide com novos comandos, knobs ou dependências.

10 Limitações e referências

10.1 Limitações conhecidas

COMO PREENCHER

Liste limitações reais do testbench, não bugs já corrigidos. Para cada limitação, explique impacto, motivo e eventual workaround.

Limitação	Impacto	Workaround / próximo passo
[Ex.: PLL é mocked]	[Não valida comportamento analógico real.]	[Fora de escopo da fase.]
[Adicionar]	[Impacto]	[Ação]

10.2 Referências

[1] Verification Plan – Fase 2.

[2] Verification Report – Fase 2.

[3] RTL / especificação do sistema de topo.

[4] Documentação do simulador e UVM utilizada no ambiente.